

Ejercicios evaluables UA7

Esta actividad aborda los criterios de evaluación del RA.

Instrucciones

- Todas las actividades que se realicen deben entregarse mediante un enlace al repositorio de Github y un documento (si se pide alguna evidencia o se plantean preguntas): DAW_DWES_UA7_ApellidosNombre.pdf
- Las entregas deberán realizarse en la tarea correspondiente en el formato especificado antes de la hora de finalización. Cualquier tarea que no cumpla estos requisitos tendrá una calificación de 0.
- Se pedirá que defiendas tu práctica. Si no puedes justificar tu trabajo o responder las preguntas que se te plantean, la calificación de la tarea será de 0.

Sobre el uso de IA y fuentes externas

Puedes utilizar herramientas como buscadores o inteligencia artificial como apoyo, pero no como sustituto de tu trabajo. Las respuestas deben estar redactadas por ti, con tus propias ideas y razonamientos.

Se valorará especialmente el análisis personal y la justificación razonada.

Ejercicio 1

Durante las semanas anteriores has estado usando métodos HTTP y códigos de estado casi sin darte cuenta.

En este ejercicio toca pararse un momento y entender por qué se usan, qué significa cada uno y si están bien aplicados en el proyecto.

La idea no es memorizar listas, sino saber justificar por qué una ruta es POST y no GET, o por qué devuelve 400 y no 404.

Qué debes hacer

- ☐ Explicar, con tus palabras: Para qué sirven GET, POST, PUT, PATCH y DELETE y en qué casos se usan dentro de este proyecto
- ☐ Explicar los códigos: 200, 201, 204, 400, 401, 403, 404, 409, 500. Qué significan y en qué situaciones aparecen en tu backend

Qué debes tener en cuenta

- ☐ No copiar definiciones sin explicarlas
- ☐ Los ejemplos deben salir del proyecto que estás haciendo

Qué debes entregar

- ☐ Documento en Markdown o PDF
- ☐ Prepararte para explicarlo brevemente en clase

Ejercicio 2

Un usuario entra en su cuenta y decide cambiar la contraseña.

El backend debe comprobar que los datos son correctos antes de hacer el cambio.

Si todo va bien, la contraseña se actualiza; si no, se devuelve un error claro.

Dónde se hace

- ☐ Ruta: POST /api/users/me/password/

Qué envía el frontend

JSON obligatorio con: *current_password* y *new_password*

Qué debe comprobar el backend

- ☐ El usuario está autenticado
- ☐ La contraseña actual es correcta
- ☐ La nueva contraseña tiene al menos 8 caracteres

Respuestas esperadas

- ☐ Todo correcto: código 200 con body: { "ok": true }
- ☐ Datos incorrectos o contraseña actual errónea: código 400 con error: validation_error (formato de semanas anteriores)
- ☐ No autenticado: código 401 con error: unauthorized (formato de semanas anteriores).

Qué debes tener en cuenta

- ☐ Nunca se devuelven contraseñas
- ☐ Un usuario solo puede cambiar su propia contraseña

Qué debes probar

- ☐ Un cambio correcto (200)
- ☐ Varios intentos inválidos (400: contraseña actual incorrecta, contraseña nueva corta, JSON vacío)
- ☐ Un intento sin autenticar (401)

Ejercicio 3

Tras varias semanas, el código empieza a crecer y a repetirse.

En este ejercicio se trata de ordenar y limpiar el backend por dentro, sin cambiar nada de lo que ve el frontend.

Dónde se hace

En todo el código de tu servidor Django.

Qué debes hacer

Detectar lógica repetida o vistas demasiado grandes y mover esa lógica a funciones auxiliares o módulos, manteniendo las mismas rutas, códigos HTTP y formatos JSON.

Qué debes probar

Todas las rutas que funcionaban antes deben seguir funcionando igual (creación, listado, detalle, actualización, usuarios...) con los mismos códigos y respuestas.

Ejercicio 4

Un usuario quiere cambiar por completo la información de un juego de su biblioteca.

El backend debe sustituir todos los datos anteriores por los nuevos.

Dónde se hace

- ☐ Ruta: PUT /api/library/entries/{id}/

Qué envía el frontend

JSON completo con *external_game_id*, *status* y *hours_played*.

Comportamiento

- ☐ El usuario debe estar autenticado y ser el dueño del recurso.
- ☐ Si falta algún campo o los datos no son válidos, se devuelve 400.
- ☐ Si no está autenticado, 401.
- ☐ Si el recurso no existe o no es suyo, 404.
- ☐ Si todo es correcto, 200 con el recurso actualizado.

Qué debes probar

- ☐ Un PUT correcto (200)
- ☐ Varios inválidos (400 por datos incorrectos)
- ☐ Un intento sin autenticar (401)
- ☐ Un intento sobre un recurso ajeno o inexistente (404).

Ejercicio 5

En las semanas anteriores ya has implementado un endpoint que permite actualizar una entrada de la biblioteca.

En este ejercicio no se trata de añadir funcionalidad nueva, sino de analizar lo que ya existe y comprobar si las decisiones tomadas son las más adecuadas.

Dónde se hace

- ☐ Actualizar: PATCH /api/library/entries/{id}/

Qué debes analizar

Debes revisar el endpoint anterior y valorar, razonándolo:

- ☐ Si el método HTTP que se está usando es el más adecuado.
- ☐ Si es correcto que el backend compruebe qué usuario hace la petición y por qué es necesario (o no).
- ☐ Si los códigos de estado que se devuelven en cada caso (200, 400, 401, 404, etc.) son adecuados.
- ☐ Si hay algún caso que no esté contemplado o alguna comprobación que debería añadirse.
- ☐ Si el comportamiento del endpoint es coherente con el resto de la API.

Qué debes hacer

- ☐ Analizar el endpoint tal y como está en tu proyecto.
- ☐ Explicar qué decisiones te parecen correctas.
- ☐ Indicar qué cambiarías, si cambiarías algo, y por qué.

Ejercicio 6

Un usuario termina de usar la plataforma y quiere salir de su cuenta.

El backend debe cerrar la sesión para que, a partir de ese momento, el usuario vuelva a estar como no autenticado.

Dónde se hace

- ☐ Ruta: POST /api/auth/logout/

Qué envía el frontend

No es necesario enviar datos.

Comportamiento esperado

- ☐ Si el usuario está autenticado, se cierra la sesión.
- ☐ Si el usuario no está autenticado, la respuesta debe ser la misma al caso anterior.

Respuesta esperada

- ☐ Código: 204
- ☐ Body: vacío.

Qué debes probar

- ☐ Realizar un login, hacer logout y comprobar que la ruta para comprobar la información del perfil devuelve 401.
- ☐ Hacer logout sin estar autenticado y comprobar que la respuesta es 204.

Ejercicio 7

Un usuario decide borrar su cuenta de la plataforma.

El backend debe permitirlo únicamente cuando la petición la hace el propio usuario.

Dónde se hace

- ☐ Ruta: DELETE /api/users/me/

Comportamiento esperado

- ☐ El usuario debe estar autenticado.
- ☐ Solo se puede borrar la cuenta del usuario que hace la petición.

Qué ocurre con la biblioteca

Al borrar la cuenta, se deben borrar también todas las entradas de biblioteca asociadas a ese usuario. No deben quedar entradas de biblioteca sin usuario.

Respuesta esperada:

- ☐ Si todo va bien, un código 204 con body vacío.
- ☐ Si no está autenticado, código 401 con error: unauthorized

Qué debes probar

- ☐ Borrar la cuenta estando autenticado devuelve 204 y, a continuación, la ruta para comprobar la información del perfil debe devolver 401.
- ☐ Intentar iniciar sesión con ese usuario ya no es posible.
- ☐ Un intento de borrado sin estar autenticado devuelve 401.

Rúbrica de evaluación de la tarea

Criterio de evaluación	Nivel alto (9-10)	Nivel medio (5-9)	Nivel Bajo (1-5)	No logrado (0)